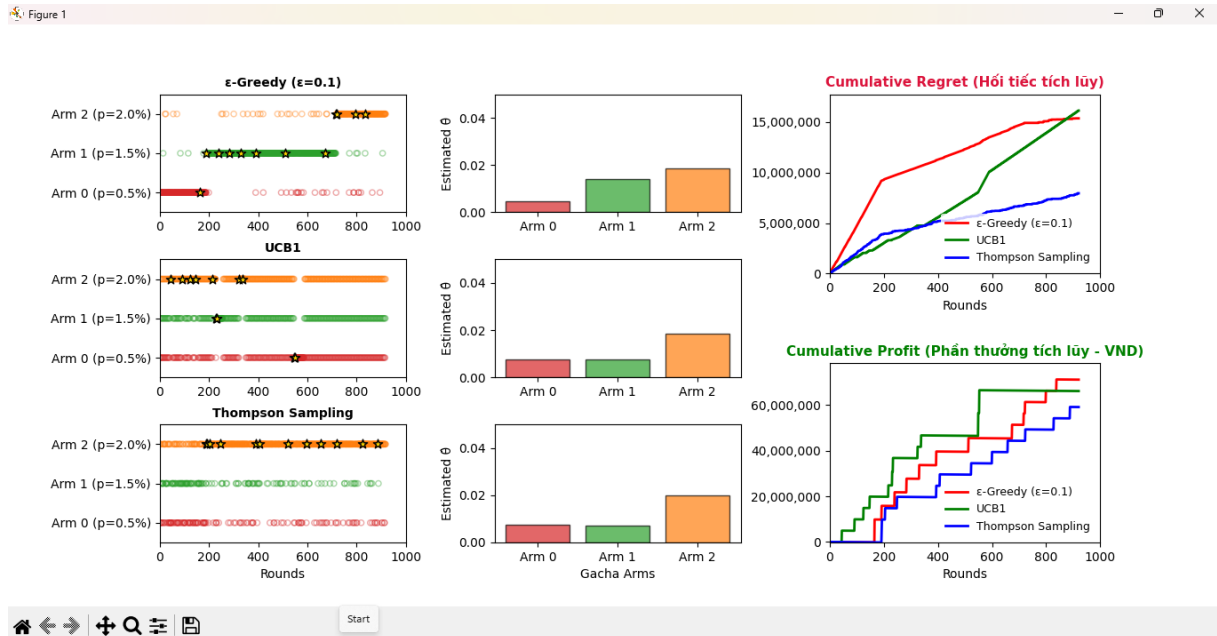


TRÍ TUỆ NHÂN TẠO – CT332

THỰC HÀNH BUỔI 2

TRẦN QUỐC TOÀN B2410699



Code: Multi-Arm Bandit

```
import matplotlib
matplotlib.use('TkAgg')
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.animation import FuncAnimation

np.random.seed(42)
true_probs = [0.005, 0.015, 0.02]
rewards = [10000000, 6000000, 5000000]
cost = 1000

n_arms = len(true_probs)
n_rounds = 1000

epsilon = 0.1
policies = ['eps_greedy', 'ucb', 'thompson']
titles = [" $\epsilon$ -Greedy ( $\epsilon=0.1$ )", "UCB1", "Thompson Sampling"]
arm_colors = ['tab:red', 'tab:green', 'tab:orange']

Q_eps = np.zeros(n_arms)
N_eps = np.zeros(n_arms)
Q_ucb = np.zeros(n_arms)
N_ucb = np.zeros(n_arms)
alpha_ts = np.ones(n_arms)
beta_ts = np.ones(n_arms)

histories = {p: [] for p in policies}
regret_history = {p: [0] for p in policies}
profit_history = {p: [0] for p in policies}

true_evs = [p * r for p, r in zip(true_probs, rewards)]
best_ev = max(true_evs)
```

```

def select_epsilon_greedy():
    if np.random.rand() < epsilon:
        return np.random.randint(n_arms)
    return np.argmax(Q_eps)

def select_ucb(t):
    if 0 in N_ucb:
        return np.argmax(N_ucb)
    ucb_values = Q_ucb + 0.3 * np.sqrt(2 * np.log(t + 1) / N_ucb)
    return np.argmax(ucb_values)

def select_thompson():
    samples = np.random.beta(alpha_ts, beta_ts)
    return np.argmax(samples)

def update_estimates(policy, a, r):
    if policy == 'eps_greedy':
        N_eps[a] += 1
        Q_eps[a] += (r - Q_eps[a]) / N_eps[a]
    elif policy == 'ucb':
        N_ucb[a] += 1
        Q_ucb[a] += (r - Q_ucb[a]) / N_ucb[a]
    elif policy == 'thompson':
        alpha_ts[a] += r
        beta_ts[a] += (1 - r)

def run_one_step(policy, t):
    if policy == 'eps_greedy':
        a = select_epsilon_greedy()
    elif policy == 'ucb':
        a = select_ucb(t)
    else:
        a = select_thompson()

    r = np.random.rand() < true_probs[a]
    update_estimates(policy, a, r)
    histories[policy].append((t, a, r))

    regret = best_ev - true_evs[a]
    regret_history[policy].append(regret_history[policy][-1] + regret)

    net_reward = rewards[a] - cost if r else -cost
    profit_history[policy].append(profit_history[policy][-1] + net_reward)

fig = plt.figure(figsize=(16, 9))
main_grid = plt.GridSpec(1, 3, width_ratios=[1, 1, 1.1], wspace=0.35)

left_grid = main_grid[0].subgridspec(3, 1, hspace=0.4)
middle_grid = main_grid[1].subgridspec(3, 1, hspace=0.4)
right_grid = main_grid[2].subgridspec(2, 1, hspace=0.5)

axes_scatter = []
axesBars = []

for i, policy in enumerate(policies):
    ax_scatter = fig.add_subplot(left_grid[i])
    ax_scatter.set_xlim(0, n_rounds)
    ax_scatter.set_ylim(-0.5, n_arms - 0.5)
    ax_scatter.set_yticks(range(n_arms))
    ax_scatter.set_yticklabels([f"Arm {j} (p={true_probs[j] * 100:.1f}%) " for
j in range(n_arms)])
    ax_scatter.set_title(titles[i], fontweight='bold', fontsize=10)

```

```

if i == 2: ax_scatter.set_xlabel("Rounds")
axes_scatter.append(ax_scatter)

ax_bar = fig.add_subplot(middle_grid[i])
ax_bar.set_ylim(0, max(true_probs) * 2.5)
ax_bar.set_xticks(range(n_arms))
ax_bar.set_xticklabels(["Arm 0", "Arm 1", "Arm 2"])
ax_bar.set_ylabel("Estimated  $\theta^*$ ")
if i == 2: ax_bar.set_xlabel("Gacha Arms")
axes_bar.append(ax_bar)

bars = [ax.bar(range(n_arms), [0] * n_arms, color=arm_colors, alpha=0.7,
edgecolor='black') for ax in axes_bar]

ax_regret = fig.add_subplot(right_grid[0])
ax_regret.set_title("Cumulative Regret (Hối tiếc tích lũy)",
fontweight='bold', color='crimson', fontsize=11)
ax_regret.set_xlim(0, n_rounds)
ax_regret.set_xlabel("Rounds")

ax_regret.ticklabel_format(style='plain', axis='y')
ax_regret.get_yaxis().set_major_formatter(plt.FuncFormatter(lambda x, loc:
"{:,}".format(int(x))))

ax_profit = fig.add_subplot(right_grid[1])
ax_profit.set_title("Cumulative Profit (Phần thưởng tích lũy - VND)",
fontweight='bold', color='green', fontsize=11)
ax_profit.set_xlim(0, n_rounds)
ax_profit.set_xlabel("Rounds")

ax_profit.ticklabel_format(style='plain', axis='y')
ax_profit.get_yaxis().set_major_formatter(plt.FuncFormatter(lambda x, loc:
"{:,}".format(int(x))))

policy_lines = {'eps_greedy': 'red', 'ucb': 'green', 'thompson': 'blue'}
lines_regret = {p: ax_regret.plot([], [], color=policy_lines[p],
label=titles[k], linewidth=2)[0] for k, p in
enumerate(policies)}
lines_profit = {p: ax_profit.plot([], [], color=policy_lines[p],
label=titles[k], linewidth=2)[0] for k, p in
enumerate(policies)}

ax_regret.legend(loc="lower right", fontsize=9, frameon=True,
facecolor='white', edgecolor='none')
ax_profit.legend(loc="lower right", fontsize=9, frameon=True,
facecolor='white', edgecolor='none')

def update(frame):
    for i, policy in enumerate(policies):
        run_one_step(policy, frame)
        t_list = [h[0] for h in histories[policy]]
        arm_list = [h[1] for h in histories[policy]]
        reward_list = [h[2] for h in histories[policy]]

        for coll in list(axes_scatter[i].collections):
            coll.remove()

        for arm in range(n_arms):
            arm_times = [t for t, a in zip(t_list, arm_list) if a == arm]
            arm_rewards = [r for a, r in zip(arm_list, reward_list) if a ==
arm]

            x_empty = [t for t, r in zip(arm_times, arm_rewards) if r == 0]
            y_empty = [arm] * len(x_empty)

```

```

        x_filled = [t for t, r in zip(arm_times, arm_rewards) if r ==
1]
        y_filled = [arm] * len(x_filled)

        axes_scatter[i].scatter(x_empty, y_empty, facecolors='none',
edgecolors=arm_colors[arm], s=20, alpha=0.4)
        if x_filled:
            axes_scatter[i].scatter(x_filled, y_filled, color='gold',
marker='*', s=60, edgecolors='black',
                                zorder=3)

        if policy == 'eps_greedy':
            est = Q_eps.copy()
        elif policy == 'ucb':
            est = Q_ucb.copy()
        else:
            est = alpha_ts / (alpha_ts + beta_ts)

        for j, b in enumerate(bars[i]):
            b.set_height(est[j])

        lines_regret[policy].set_data(range(len(regret_history[policy])),
regret_history[policy])
        lines_profit[policy].set_data(range(len(profit_history[policy])),
profit_history[policy])

        all_regrets = [val for p in policies for val in regret_history[p]]
        all_profits = [val for p in policies for val in profit_history[p]]

        ax_regret.set_ylim(min(all_regrets) - 5, max(all_regrets) * 1.1 + 10)
        ax_profit.set_ylim(min(all_profits) * 1.1 - 2000, max(all_profits) *
1.1 + 10000)

        return list(lines_regret.values()) + list(lines_profit.values()) + [b
for bar in bars for b in bar]

anim = FuncAnimation(fig, update, frames=n_rounds, interval=15, blit=False,
repeat=False)
plt.show()

```

--HÉT--